

Trabalho de Implementação 1:

Cifra de Vigenère — Implementação e Criptoanálise

Rafael de Lima Pereira
Matrícula: 242043277

Pedro Freitas França
Matrícula: 232006028

Rian Kallebe da Silva Lisboa
Matrícula: 242012000

Arthur Martins Pereira de Souza
Matrícula: 241004499

Departamento de Ciência da Computação
Universidade de Brasília

Resumo—Este relatório descreve a implementação de um cifrador/decifrador da cifra de Vigenère e o desenvolvimento de um ataque de recuperação de chave baseado em propriedades estatísticas da linguagem. São apresentadas as decisões de projeto quanto ao tratamento do alfabeto, a metodologia de criptoanálise adotada, os resultados obtidos sobre os criptogramas fornecidos e uma análise das limitações da abordagem.

Index Terms—cifra de Vigenère, criptoanálise, índice de coincidência, análise de frequência

I. INTRODUÇÃO

A cifra de Vigenère, descrita no século XVI, é uma cifra de substituição polialfabética que durante cerca de três séculos foi considerada indecifrável, o que lhe rendeu a alcunha de *le chiffre indéchiffrable* [1]. Sua segurança repousa na ideia de que uma mesma letra do texto claro pode ser mapeada em letras diferentes do criptograma, dependendo da posição em que ocorre, o que achata a distribuição de frequências que denuncia as cifras monoalfabéticas.

Esse achatamento, contudo, é apenas parcial: a chave se repete ciclicamente e, uma vez conhecido o seu comprimento, o criptograma se decompõe em vários textos cifrados por cifra de César, cada um deles vulnerável à análise de frequência clássica.

Este trabalho está dividido em duas partes complementares. A Parte I consiste na implementação de um cifrador e decifrador, discutida na Seção III. A Parte II consiste na criptoanálise de dois criptogramas de chave desconhecida — um em português e outro em inglês — tratada nas Seções IV e V. O código-fonte completo está disponível em [4].

II. A CIFRA DE VIGENÈRE

Seja $\Sigma = \{A, B, \dots, Z\}$ o alfabeto de $n = 26$ letras, e seja cada letra identificada com um inteiro em $\{0, 1, \dots, 25\}$. Dada uma mensagem $P = p_0 p_1 \dots p_{m-1}$ e uma chave $K = k_0 k_1 \dots k_{\ell-1}$ de comprimento ℓ , o criptograma $C = c_0 c_1 \dots c_{m-1}$ é obtido por

$$c_i = (p_i + k_{i \bmod \ell}) \bmod n, \quad (1)$$

e a decifração, pela operação inversa,

$$p_i = (c_i - k_{i \bmod \ell}) \bmod n. \quad (2)$$

A cifra pode ser vista, portanto, como a aplicação intercalada de ℓ cifras de César independentes: todas as posições i congruentes entre si módulo ℓ são deslocadas pelo mesmo valor $k_{i \bmod \ell}$. Essa observação é simultaneamente a descrição do algoritmo e o ponto de partida do ataque descrito na Seção IV.

Vale notar que para $\ell = 1$ a cifra degenera na cifra de César, e que para $\ell \geq m$ com chave aleatória e de uso único obtém-se o cifrador de Vernam, comprovadamente seguro. A fragilidade da construção está exatamente na faixa intermediária, de uso prático: ℓ pequeno em relação a m e chave dotada de estrutura linguística.

III. DECISÕES DE IMPLEMENTAÇÃO (PARTE I)

A implementação foi feita em Python 3, sem qualquer biblioteca de criptografia: as Equações (1) e (2) estão codificadas diretamente no módulo `vigenere/cipher.py`. O projeto é organizado em quatro partes: o núcleo da cifra (`cipher.py`), o ataque (`attack.py`), a interface de linha de comando (`cli.py`) e a suíte de testes automatizados (`tests/`).

A. Núcleo da cifra

O laço central percorre a mensagem caractere a caractere, mantendo um índice próprio para a chave, e é reproduzido de forma resumida a seguir:

```
for char in text:
    letter = fold(char)
    if letter is None:
        # Fora do alfabeto: mantido tal e qual
        # (sem avançar a chave) no modo "preserve",
        # descartado no modo "strict".
        if alphabet == "preserve":
            result.append(char)
        continue

    # C_i = (P_i + K_i) mod 26
    # P_i = (C_i - K_i) mod 26
    base = ord("A") if letter.isupper() else ord("a")
    shift = offsets[key_index % len(offsets)] * sign
    shifted = chr((ord(letter) - base + shift)
                  % ALPHABET_SIZE + base)
    result.append(shifted)
```

```
key_index += 1
```

O parâmetro `sign` vale `+1` na cifração e `-1` na decifração, de modo que uma única rotina implementa as duas equações. Como `key_index` é incrementado apenas quando um caractere é efetivamente cifrado, a chave nunca perde a sincronia com as letras às quais se aplica — propriedade indispensável para que a decifração seja o inverso exato da cifração.

B. Tratamento do alfabeto

O alfabeto adotado é o clássico de 26 letras. A decisão de projeto mais relevante diz respeito aos caracteres que não pertencem a ele. A Tabela I resume o tratamento aplicado a cada classe de caractere.

Tabela I
TRATAMENTO DE CADA CLASSE DE CARACTERE

Entrada	Tratamento
a-z / A-Z	Cifradas. Caixa preservada no modo <code>preserve</code> ; forçada a maiúscula no modo <code>strict</code> .
Letras acentuadas (á â ã ä å ç è é ê í ò ô õ ö ü ù ñ)	Reduzidas à letra base por decomposição Unicode NFD e então cifradas. O acento não é restaurado na decifração.
Espaços, dígitos, pontuação	<code>preserve</code> : copiados sem alteração, sem consumir letra da chave. <code>strict</code> : descartados.
Caracteres da chave	Reduzidos da mesma forma; o que não é letra é ignorado. Chave sem nenhuma letra é erro.

1) *Redução de acentos por NFD*: Textos em português contêm letras acentuadas que não pertencem a Σ . Duas alternativas foram consideradas: preservá-las intactas no criptograma, ou reduzi-las à letra base. Optou-se pela segunda, implementada pela normalização Unicode NFD, que decompõe ç em c seguido de uma cedilha combinante, bastando então tomar o primeiro ponto de código:

```
def fold(char):  
    base = unicodedata.normalize("NFD", char)[1:]  
    if "a" <= base <= "z" or "A" <= base <= "Z":  
        return base  
    return None
```

A justificativa é criptográfica. Preservar os acentos faria com que ç, ã e é aparecessem em claro no criptograma, o que não apenas vazaria informação sobre o texto original como praticamente identifica o idioma e restringe fortemente as palavras possíveis. A redução é ainda mais robusta que uma lista fixa de caracteres acentuados: como opera sobre a decomposição Unicode, trata corretamente qualquer letra latina acentuada, inclusive as que não ocorrem em português. O custo dessa escolha é a perda de informação: ação é recuperado como acao. Trata-se de perda ortográfica, e não semântica, e é o comportamento usual em criptoanálise clássica.

2) *Modos de alfabeto*: Foram implementados dois modos, selecionáveis pela opção `--alphabet`:

- `preserve` (padrão): espaços, dígitos e pontuação são copiados sem alteração e não consomem letra da chave; a caixa das letras é mantida. O resultado permanece legível,

mas a estrutura do texto claro (limites de palavras e pontuação) continua visível no criptograma.

- `strict`: tudo o que não é letra é descartado e a saída é uma sequência contínua de letras maiúsculas. É o formato canônico dos exercícios de criptoanálise, e é o modo empregado na Parte II, pois não revela limites de palavras.

A existência do modo `strict` tem, além disso, papel metodológico: ela permite gerar criptogramas no mesmo formato dos fornecidos pelo enunciado, o que possibilita validar o ataque contra textos de chave conhecida antes de aplicá-lo aos criptogramas do trabalho.

C. Validação das entradas

A única condição que impede a execução é uma chave sem nenhuma letra do alfabeto, que provoca erro explícito. As demais situações suspeitas geram avisos na saída de erro padrão, sem interromper o processamento:

- chave da qual foram removidos caracteres ou cujos acentos foram reduzidos — é exibida a chave efetivamente utilizada, evitando que o usuário decifre com chave diferente da que julga estar usando;
- chave de uma única letra, equivalente à cifra de César;
- texto sem nenhuma letra do alfabeto, caso em que nada é cifrado.

A opção `-v` exibe um resumo dos parâmetros efetivos (comando, alfabeto, chave utilizada e seu tamanho, número de caracteres e de letras da entrada), e as opções `-i/-o` permitem ler e escrever arquivos, necessárias para tratar os criptogramas da Parte II.

D. Verificação

A implementação é acompanhada de uma suíte de 26 testes automatizados (`pytest`) que cobrem: o vetor clássico ATTACKATDAWN/LEMON $\rightarrow$ LXFOPEVFRNHR; a inversibilidade da cifra; a preservação da caixa; a redução de acentos e a consequente sincronia da chave; o comportamento dos dois modos de alfabeto; o tratamento de chaves com caracteres inválidos; e o comportamento da interface de linha de comando, incluindo os avisos e as mensagens de erro.

IV. METODOLOGIA DO ATAQUE (PARTE II)

A quebra de uma cifra polialfabética como a de Vigenère baseia-se na exploração das propriedades estatísticas das linguagens naturais. Diferentemente de sequências puramente aleatórias, idiomas como o português e o inglês possuem distribuições de frequência de letras desiguais e previsíveis. O ataque explora ainda o fato, observado na Seção II, de que a cifra é a intercalação de ℓ cifras de César. A abordagem fundamenta-se em duas métricas — o índice de coincidência (IC) e o teste de aderência qui-quadrado (χ^2) — e é conduzida em duas etapas: primeiro estima-se ℓ , depois recupera-se cada letra da chave independentemente. A implementação foi feita de forma modular em Python, reutilizando o decifrador da Parte I.

A. Índice de coincidência

O IC, introduzido por Friedman [3], é a probabilidade de que dois elementos selecionados aleatoriamente de um texto sejam idênticos. Para um texto de N letras com n_i ocorrências da i -ésima letra do alfabeto,

$$IC = \frac{\sum_{i=A}^Z n_i(n_i - 1)}{N(N - 1)}. \quad (3)$$

Para um texto aleatório uniforme o valor esperado é $1/26 \approx 0,038$; para textos em linguagem natural situa-se em torno de 0,066 em inglês e 0,074 em português, por causa da distribuição desigual das letras. Essa desigualdade é o fundamento estatístico de todo o ataque: a Tabela II apresenta as frequências relativas adotadas na implementação [2], as mesmas usadas pelo teste χ^2 na recuperação da chave.

Tabela II
FREQUÊNCIA RELATIVA DAS LETRAS (%) EM PORTUGUÊS E EM INGLÊS

Letra	PT	EN	Letra	PT	EN
A	14,63	8,17	N	5,05	6,75
B	1,04	1,49	O	10,73	7,51
C	3,88	2,78	P	2,52	1,93
D	4,99	4,25	Q	1,20	0,10
E	12,57	12,70	R	6,53	5,99
F	1,02	2,23	S	7,81	6,33
G	1,30	2,02	T	4,34	9,06
H	1,28	6,09	U	4,63	2,76
I	6,18	6,97	V	1,67	0,98
J	0,40	0,15	W	0,01	2,36
K	0,02	0,77	X	0,21	0,15
L	2,78	4,03	Y	0,01	1,98
M	4,74	2,41	Z	0,47	0,07

Observa-se, por exemplo, que A e E dominam o português (14,63% e 12,57%), enquanto no inglês E e T são as mais frequentes (12,70% e 9,06%). Letras como K, W e Y são praticamente ausentes em português. Essa assimetria é o que o índice de coincidência detecta globalmente e o que o qui-quadrado explorará localmente em cada coluna.

B. Estimativa do comprimento da chave

A estimativa é automatizada iterando sobre comprimentos candidatos $k = 1, 2, \dots, 20$. Para cada k , o criptograma é subdividido em k colunas — cada coluna contém os caracteres cifrados pela mesma posição da chave — e calcula-se a média do IC dessas colunas. Quando k é o comprimento correto (ou um múltiplo dele), cada coluna resulta de uma única cifra de César, que é uma permutação do alfabeto e portanto *preserva* o IC do idioma original: o IC médio aproxima-se de 0,07. Quando k está errado, as colunas misturam deslocamentos distintos e o IC médio permanece próximo de 0,038. Seleciona-se o k que maximiza o IC médio, pois indica que os caracteres de cada coluna foram deslocados pelo mesmo caractere da chave.

C. Análise de frequência e recuperação da chave

Fixado k , o texto permanece particionado em seus k subconjuntos. Para cada coluna, aplica-se força bruta sobre os 26 deslocamentos possíveis da cifra de César: a coluna é decifrada

temporariamente e a distribuição resultante é comparada à distribuição esperada do idioma (português ou inglês) pela estatística qui-quadrado

$$\chi^2 = \sum_{i=A}^Z \frac{(O_i - E_i)^2}{E_i}, \quad (4)$$

em que O_i é o número de ocorrências observadas da letra i e E_i é o número esperado, calculado a partir do tamanho da coluna e da probabilidade teórica da letra no idioma (Tabela II). O deslocamento que minimiza χ^2 — a menor divergência em relação ao idioma alvo — é o candidato mais provável para aquela posição da chave. A concatenação desses caracteres reconstrói a chave completa e permite decifrar o criptograma.

D. Avaliação e refinamento

A chave candidata é usada para decifrar o criptograma, e o resultado é avaliado pelo IC do texto decifrado — que deve aproximar-se do valor esperado do idioma — e por inspeção da legibilidade. Quando alguma coluna é curta demais para que a estatística seja confiável, o segundo e o terceiro melhores candidatos de χ^2 daquela posição são examinados, refinando-se a chave posição a posição.

V. RESULTADOS

A. Parte I

O cifrador reproduz o vetor de teste clássico da literatura e é inversível sobre texto em português, como ilustrado a seguir:

```
$ vigenere encode "ATTACKATDAWN" --key LEMON
LXFOPVEFRNHR
$ vigenere decode "LXFOPVEFRNHR" --key LEMON
ATTACKATDAWN
$ vigenere encode "Ataque ao amanhecer!" \
--key segredo --alphabet strict
SXGHYHOGESRRKSUIX
$ vigenere decode "SXGHYHOGESRRKSUIX" \
--key segredo --alphabet strict
ATAQUEAOAMANHECER
```

Note-se, no último exemplo, o efeito documentado do modo `strict`: a pontuação e os espaços não são recuperados, pois foram descartados na cifração. A suíte de 26 testes automatizados é executada integralmente com sucesso.

B. Parte II

Dois criptogramas de validação — um em português (762 letras) e outro em inglês (697 letras), cifrados no modo `strict` com chaves distintas — foram processados pelo comando `vigenere attack`. O ataque reproduz, na prática, as duas etapas da Seção IV: primeiro estima o comprimento da chave pelo IC médio das colunas; depois recupera cada letra pelo mínimo de χ^2 frente às frequências da Tabela II.

1) *Estimativa do comprimento da chave pelo IC*: Para cada criptograma, o módulo percorreu $k \in \{1, \dots, 20\}$, particionou o texto em k colunas e calculou o IC médio. A Tabela III destaca os comprimentos mais informativos. No criptograma em português, o IC médio salta de valores próximos de 0,044 (próximos do aleatório) para 0,078 em $k = 9$, valor

compatível com linguagem natural; o segundo pico aparece em $k = 18$ (0,077), múltiplo do comprimento correto. No inglês, o máximo ocorre em $k = 6$ (0,068), com picos secundários em $k = 12$ e $k = 18$. Em ambos os casos o algoritmo seleciona automaticamente o k de maior IC médio — 9 e 6, respectivamente — sem intervenção manual.

Tabela III
IC MÉDIO DAS COLUNAS PARA COMPRIMENTOS SELECIONADOS

Criptograma	$k=1$	$k=6$	$k=9$	$k=12$	$k=18$
Português	0,044	0,052	0,078	0,052	0,077
Inglês	0,044	0,068	0,053	0,068	0,068

Esse comportamento confirma a intuição teórica: só quando k coincide com o período da chave (ou com um múltiplo) cada coluna resulta de uma única cifra de César e o IC médio se eleva ao patamar do idioma; caso contrário, as colunas misturam deslocamentos distintos e o IC permanece baixo.

2) *Recuperação da chave pela análise de frequência*: Fixado k , cada coluna é tratada como um César independente. Para cada uma das 26 hipóteses de deslocamento, a coluna é temporariamente decifrada e a distribuição resultante é comparada à da Tabela II via χ^2 (4). O deslocamento de menor χ^2 é a letra candidata daquela posição. A Tabela IV ilustra os três melhores candidatos em posições representativas: a margem entre o primeiro e o segundo lugar é ampla (por exemplo, $\chi^2 \approx 22$ contra ≈ 749 na primeira posição do português), o que torna a escolha inequívoca.

Tabela IV
CANDIDATOS DE χ^2 POR POSIÇÃO (LETRA: VALOR; MENOR É MELHOR)

Pos.	PT ($k=9$)	EN ($k=6$)
0	S:22,1; T:749; F:758	C:25,7; P:374; I:386
1	E:14,0; S:813; P:919	I:20,6; B:542; O:579
2	G:21,7; T:204; H:398	P:32,0; F:422; A:473
3	U:17,5; F:514; H:694	H:49,8; U:237; A:250
4	R:26,5; C:547; Q:725	E:25,3; R:275; K:343
5	A:24,4; L:402; N:802	R:20,8; E:244; X:342

Concatenando o melhor candidato de cada posição obtém-se SEGURANCA (português) e CIPHER (inglês). Em nenhuma posição foi necessário recorrer ao segundo ou ao terceiro colocado: o mínimo de χ^2 coincidiu com a letra correta em todas as colunas. A Tabela V resume o desfecho. Após a decifração, o IC do texto claro aproximou-se do valor esperado do idioma (0,079 frente a $\approx 0,074$ em português; 0,067 frente a $\approx 0,066$ em inglês).

Tabela V
RESULTADOS DA RECUPERAÇÃO DE CHAVE

Criptograma	k estimado	Chave recuperada	IC final
Português	9	SEGURANCA	0,079
Inglês	6	CIPHER	0,067

VI. LIMITAÇÕES DA ABORDAGEM

O ataque descrito é estatístico e, como tal, depende de premissas que nem sempre se verificam.

Volume de texto. Tanto o IC quanto o qui-quadrado são estimadores que convergem lentamente. Cada subconjunto tem cerca de N/k letras; quando essa razão cai abaixo de algumas dezenas de caracteres, a distribuição observada torna-se ruidosa e a letra correta da chave pode não minimizar χ^2 . O ataque degrada, portanto, para criptogramas curtos ou chaves longas, e falha por completo no caso limite $k \geq N$.

Dependência do idioma. O método pressupõe conhecimento prévio da distribuição de frequências do idioma do texto claro. Textos técnicos, com vocabulário atípico, ou textos que misturam idiomas produzem distribuições distintas das tabelas de referência. Neste trabalho o idioma é informado na execução do ataque (`--language pt|en`); num cenário real seria necessário testar as duas hipóteses e escolher a de melhor ajuste.

Efeito do pré-processamento. A redução de acentos altera levemente as frequências do português — ocorrências de ç são somadas às de c, e as vogais acentuadas às suas bases. Como as tabelas de referência usuais já são construídas sobre o alfabeto não acentuado, o efeito é pequeno, mas introduz um viés que não é nulo.

Múltiplos do comprimento. A estimativa por IC não distingue k de seus múltiplos, já que particionar por $2k$ também isola cifras de César e pode maximizar (ou empatar) o IC médio. Nos casos observados, a inspeção dos valores de IC ao longo de $k = 1, \dots, 20$ e a legibilidade do texto decifrado permitem discriminar o comprimento verdadeiro do múltiplo.

Escopo da implementação. O cifrador opera sobre um alfabeto de 26 letras: dígitos e pontuação não são cifrados, apenas preservados ou descartados. Uma mensagem cujo conteúdo relevante seja numérico não é protegida pela implementação.

VII. CONCLUSÃO

A implementação da cifra de Vigenère é direta: as decisões críticas estão no alfabeto e no tratamento dos caracteres fora dele, com consequências criptográficas concretas. A criptoanálise confirma que a repetição periódica da chave é o ponto de ruptura — permite decompor o problema em subcifras independentes, de modo que a segurança depende da razão entre o comprimento da chave e o do texto, não do tamanho do espaço de chaves.

REFERÊNCIAS

- [1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. Upper Saddle River, NJ, USA: Pearson, 2017.
- [2] Wikipédia, “Frequência de letras.” [Online]. Disponível: https://pt.wikipedia.org/wiki/Frequência_de_letras
- [3] W. F. Friedman, *The Index of Coincidence and Its Applications in Cryptography*. Geneva, IL, USA: Riverbank Laboratories, 1922.
- [4] R. Lima, “Implementação da cifra de Vigenère.” [Online]. Disponível: <https://github.com/RafaelLime/vigenere>